Building AI Agents on the Microsoft Stack:

Microsoft Agent Framework (MAF) and Microsoft Foundry Agent Service

Hobhav Fulzele

November 10, 2026

Contents

1 Introduction **2**

2 What is Microsoft Agent Framework (MAF)? **2**
- 2.1 Definition . . . 2
- 2.2 Why it exists . . . 2
- 2.3 Core building blocks . . . 2
- 2.4 Open standards and interoperability . . . 3
- 2.5 When to use MAF . . . 3

3 What is Microsoft Foundry Agent Service? **3**
- 3.1 Definition . . . 3
- 3.2 How you build agents on it . . . 4
- 3.3 Kinds of agents . . . 4
- 3.4 Key capabilities . . . 4
- 3.5 When to use Foundry Agent Service . . . 4

1 Introduction

An **AI agent** is a program that uses generative AI to interpret input, decide what to do, and perform tasks for a user or for another program. Unlike a plain chatbot, an agent can call *tools* (functions, APIs, search indexes, code interpreters), keep *state* across turns, and collaborate with other agents.

Building such systems raises two distinct sets of questions:

1. **Design and implementation:** How do I define an agent, give it tools, manage conversation state, and orchestrate several agents?
2. **Operation:** Where does the agent run, how is it secured, scaled, monitored, evaluated, and governed in an enterprise?

Microsoft Agent Framework answers the first set of questions; Microsoft Foundry Agent Service answers the second. The two are complementary rather than competing.

2 What is Microsoft Agent Framework (MAF)?

2.1 Definition

Microsoft Agent Framework is an open-source SDK for building AI agents and multi-agent workflows in **Python** and **.NET**. It is the direct successor to two earlier Microsoft projects, *Semantic Kernel* and *AutoGen*, and was created by the same teams.

In one sentence

MAF combines AutoGen's simple agent abstractions with Semantic Kernel's enterprise features (state management, type safety, middleware, telemetry) and adds explicit, graph-based multi-agent orchestration on top.

2.2 Why it exists

Before MAF, developers had to choose between AutoGen (easy multi-agent experimentation, research-oriented) and Semantic Kernel (enterprise-ready, but with more ceremony). MAF unifies the two into a single framework, so the same code can move from prototype to production without switching libraries.

2.3 Core building blocks

MAF is organised around two primary capabilities: **Agents** and **Workflows**.

2.3.1 Agents

An agent wraps an LLM together with instructions and tools. It processes input, decides whether to call tools or MCP servers, and generates a response. Key concepts include:

- **Chat clients / providers:** MAF is model-agnostic. Agents can be backed by Microsoft Foundry, Azure OpenAI, OpenAI, Anthropic, Ollama, Microsoft Copilot Studio, and other providers.
- **Tools:** Custom Python/C# functions, hosted tools, and tools exposed through the Model Context Protocol (MCP) or OpenAPI.
- **Conversations (threads):** Multi-turn state that lets an agent remember earlier messages.

- **Memory and persistence:** Mechanisms for storing context beyond a single session.
- **Middleware:** Hooks to intercept and modify agent behaviour (logging, filtering, guardrails).

2.3.2 Workflows

Workflows model multi-step processes as **directed graphs**. Nodes, called *executors*, may be agents or ordinary functions; *edges* define how data and control flow between them. Workflows give developers explicit control over execution order, which is useful when a task must follow a known process.

MAF inherits AutoGen's orchestration patterns:

Pattern	Behaviour
Sequential	Agents run one after another, each passing output to the next.
Concurrent	Several agents work in parallel; results are aggregated.
Handoff	One agent transfers control of the conversation to another.
Group chat	Agents collaborate in a shared conversation.
Magentic	A manager agent plans and coordinates specialist agents.

Workflows also support **checkpointing** (saving and resuming state) and **human-in-the-loop** steps such as approvals, which makes them suitable for long-running business processes.

2.4 Open standards and interoperability

MAF is built around open protocols:

- **MCP (Model Context Protocol)** – for connecting agents to data sources and tools.
- **A2A (Agent-to-Agent)** – for communication between agents, including agents built on other frameworks.
- **OpenAPI** – for calling existing REST APIs as tools.

It also integrates with the Microsoft 365 Agents SDK and with Microsoft Foundry Agent Service for enterprise deployment.

2.5 When to use MAF

- Teams standardised on Python or .NET who want one unified agent library.
- Projects that need explicit, inspectable multi-agent orchestration.
- Long-running processes requiring checkpoints or human approval.
- Applications that must remain portable across model providers and hosting environments.

3 What is Microsoft Foundry Agent Service?

3.1 Definition

Microsoft Foundry Agent Service is a fully managed Azure service for building, deploying, and scaling AI agents without managing the underlying compute and storage. It is part of **Microsoft Foundry**, the platform formerly known as *Azure AI Foundry* (and earlier, *Azure AI Studio*); the service itself was previously called *Azure AI Agent Service*.

In one sentence

Foundry Agent Service is the managed runtime and control plane that hosts, secures, observes, and publishes agents inside Microsoft Foundry.

3.2 How you build agents on it

Agents can be created in two ways:

- **Portal (low-code):** Configure instructions, choose a model, add tools, and test in a playground in the Foundry portal.
- **Code-first:** Use the Foundry SDKs (Python, C#, JavaScript, Java) or REST APIs. The newer agents API is aligned with the OpenAI Responses API, so existing Responses-style code carries over closely.

3.3 Kinds of agents

Agent type	Description
Prompt agents	Defined declaratively by instructions, a model, and tools; the service runs the reasoning loop.
Workflow agents	Multi-step or multi-agent processes orchestrated by the service.
Hosted agents	Your own agent code—written with MAF, LangGraph, or another framework—packaged and run on Foundry's managed runtime (offered in preview at the time of writing).

3.4 Key capabilities

- **Model choice:** Access to models from the Foundry model catalog.
- **Built-in tools:** File search, code interpreter, web grounding, Azure AI Search, function calling, OpenAPI tools, and MCP servers.
- **State management:** Conversation threads and storage handled by the service.
- **Identity and security:** Microsoft Entra ID, role-based access control (RBAC), audit logs, conditional access, and private networking options.
- **Safety:** Content filtering and responsible-AI controls.
- **Observability and evaluation:** Tracing, monitoring, and evaluation tooling to measure agent quality.
- **Publishing:** Endpoints for applications and channels such as Microsoft 365 and Teams.

3.5 When to use Foundry Agent Service

- You want agents running in production without operating servers.
- Governance, compliance, and identity integration are requirements.
- You need central monitoring, evaluation, and cost control for many agents across an organisation.